RES POLYTECHNIC, BANGALORE-91

CIE Practice Test IV - Model Answers

Data Structure with Python | Course Code 25CS33I | Marks 50 | Semester III

Answer any one full question from each section. Both options are answered below.

SECTION I

Q1 a) Stack using a class – LIFO (12 marks)

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)
        print(item, "pushed")

    def pop(self):
        if self.is_empty():
            print("Stack is empty")
        else:
            print(self.items.pop(), "popped")

    def peek(self):
        if self.is_empty():
            print("Stack is empty")
        else:
            print("Top element =", self.items[-1])

    def is_empty(self):
        return len(self.items) == 0

    def display(self):
        if self.is_empty():
            print("Stack is empty")
        else:
            print("Stack (bottom to top):", self.items)

s = Stack()
for x in [10, 20, 30, 40]:
    s.push(x)
s.pop()
s.pop()
s.display()
```

Output:

```
10 pushed
20 pushed
30 pushed
40 pushed
40 popped
30 popped
Stack (bottom to top): [10, 20]
```

Q1 b) Menu-driven pile of plates (13 marks)

```
class Stack:                        # same class as in (a), messages changed
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def pop(self):
        return self.items.pop() if self.items else None
    def peek(self):
        return self.items[-1] if self.items else None
    def is_empty(self):
        return len(self.items) == 0

plates = Stack()
count = 0
while True:
    print("\n1.Add plate  2.Take plate  3.See top plate  4.Display  5.Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        count += 1
        plates.push("Plate " + str(count))
```

```
        print("Plate added")
    elif ch == 2:
        p = plates.pop()
        print(p, "taken" if p else "- No plates available")
    elif ch == 3:
        p = plates.peek()
        print("Top plate:", p if p else "No plates available")
    elif ch == 4:
        if plates.is_empty():
            print("No plates available")
        else:
            print("Plates (top first):", plates.items[::-1])
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

Stack = pile of plates: the last plate placed is the first taken (LIFO).

OR

Q2 a) Queue using a class – FIFO (12 marks)

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)
        print(item, "enqueued")

    def dequeue(self):
        if self.is_empty():
            print("Queue is empty")
        else:
            print(self.items.pop(0), "dequeued")

    def is_empty(self):
        return len(self.items) == 0

    def display(self):
        if self.is_empty():
            print("Queue is empty")
        else:
            print("Queue (front to rear):", self.items)

q = Queue()
for x in [10, 20, 30, 40]:
    q.enqueue(x)
q.dequeue()
q.dequeue()
q.display()
```

Output: `10 dequeued`, `20 dequeued`, `Queue (front to rear): [30, 40]`

Q2 b) Menu-driven ticket counter queue (13 marks)

```
class Queue:
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        self.items.append(item)
    def dequeue(self):
        return self.items.pop(0) if self.items else None
    def is_empty(self):
        return len(self.items) == 0

counter = Queue()
while True:
    print("\n1.Add customer  2.Serve customer  3.Display waiting  4.Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        name = input("Customer name: ")
        counter.enqueue(name)
        print(name, "joined the queue")
    elif ch == 2:
        c = counter.dequeue()
        print("Served:", c) if c else print("No customer is waiting")
    elif ch == 3:
        if counter.is_empty():
            print("No customer is waiting")
```

```
        else:
            print("Waiting customers:", counter.items)
    elif ch == 4:
        break
    else:
        print("Invalid choice")
```

Customers are served in the order they arrive (FIFO).

SECTION II

Q3 a) Singly Linked List (12 marks)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        n = Node(data)
        n.next = self.head
        self.head = n

    def insert_end(self, data):
        n = Node(data)
        if self.head is None:
            self.head = n
            return
        t = self.head
        while t.next:
            t = t.next
        t.next = n

    def insert_middle(self, data, pos):      # pos = 1 means after 1st node
        t = self.head
        for _ in range(pos - 1):
            if t is None:
                break
            t = t.next
        if t is None:
            print("Invalid position")
            return
        n = Node(data)
        n.next = t.next
        t.next = n

    def delete_begin(self):
        if self.head:
            self.head = self.head.next
        else:
            print("List is empty")

    def delete_end(self):
        if self.head is None:
            print("List is empty")
        elif self.head.next is None:
            self.head = None
        else:
            t = self.head
            while t.next.next:
                t = t.next
            t.next = None

    def delete_middle(self, key):            # delete node having value key
        t, prev = self.head, None
        while t and t.data != key:
            prev, t = t, t.next
        if t is None:
            print(key, "not found")
        elif prev is None:
            self.head = t.next
        else:
            prev.next = t.next

    def display(self):
        t = self.head
```

```
        while t:
            print(t.data, end=" -> ")
            t = t.next
        print("None")

    def search(self, key):
        t, pos = self.head, 1
        while t:
            if t.data == key:
                print(key, "found at position", pos)
                return
            t, pos = t.next, pos + 1
        print(key, "not found")

ll = SinglyLinkedList()
ll.insert_begin(20); ll.insert_begin(10)
ll.insert_end(40)
ll.insert_middle(30, 2)          # 10 -> 20 -> 30 -> 40
ll.display()
ll.search(30)
ll.delete_begin(); ll.delete_end(); ll.delete_middle(30)
ll.display()
```

Output:

```
10 -> 20 -> 30 -> 40 -> None
30 found at position 3
20 -> None
```

Q3 b) Doubly Linked List (13 marks)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        n = Node(data)
        n.next = self.head
        if self.head:
            self.head.prev = n
        self.head = n

    def insert_end(self, data):
        n = Node(data)
        if self.head is None:
            self.head = n
            return
        t = self.head
        while t.next:
            t = t.next
        t.next = n
        n.prev = t

    def insert_middle(self, data, pos):      # after pos-th node
        t = self.head
        for _ in range(pos - 1):
            if t is None:
                break
            t = t.next
        if t is None:
            print("Invalid position")
            return
        n = Node(data)
        n.next, n.prev = t.next, t
        if t.next:
            t.next.prev = n
        t.next = n

    def delete(self, key):                   # begin / middle / end by value
        t = self.head
        while t and t.data != key:
            t = t.next
        if t is None:
            print(key, "not found")
            return
```

```
        if t.prev:
            t.prev.next = t.next
        else:
            self.head = t.next
        if t.next:
            t.next.prev = t.prev

    def forward(self):
        t = self.head
        while t:
            print(t.data, end=" <-> ")
            t = t.next
        print("None")

    def backward(self):
        t = self.head
        while t and t.next:
            t = t.next
        while t:
            print(t.data, end=" <-> ")
            t = t.prev
        print("None")

d = DoublyLinkedList()
d.insert_begin(20); d.insert_begin(10); d.insert_end(40)
d.insert_middle(30, 2)
d.forward(); d.backward()
d.delete(10); d.delete(40); d.delete(30)     # begin, end, middle
d.forward()
```

Output:

```
10 <-> 20 <-> 30 <-> 40 <-> None
40 <-> 30 <-> 20 <-> 10 <-> None
20 <-> None
```

OR

Q4 a) Binary tree – search, delete leaf, recursive traversals (12 marks)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    else:
        root.right = insert(root.right, data)
    return root

def search(root, key):
    if root is None:
        return False
    if root.data == key:
        return True
    return search(root.left, key) if key < root.data else search(root.right, key)

def delete_leaf(root, key):
    if root is None:
        print("Node not found")
        return None
    if root.data == key:
        if root.left is None and root.right is None:
            print("Leaf", key, "deleted")
            return None
        print("Not a leaf node - cannot delete")
        return root
    if key < root.data:
        root.left = delete_leaf(root.left, key)
    else:
        root.right = delete_leaf(root.right, key)
    return root

def inorder(r):
    if r:
        inorder(r.left); print(r.data, end=" "); inorder(r.right)
```

```
def preorder(r):
    if r:
        print(r.data, end=" "); preorder(r.left); preorder(r.right)

def postorder(r):
    if r:
        postorder(r.left); postorder(r.right); print(r.data, end=" ")

root = None
for v in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, v)

key = int(input("Enter value to search: "))
print("Found" if search(root, key) else "Not found")

key = int(input("Enter leaf value to delete: "))
root = delete_leaf(root, key)

print("In-order  :", end=" "); inorder(root)
print("\nPre-order :", end=" "); preorder(root)
print("\nPost-order:", end=" "); postorder(root)
```

Output (search 60, delete leaf 20):

```
Found
Leaf 20 deleted
In-order  : 30 40 50 60 70 80
Pre-order : 50 30 40 70 60 80
Post-order: 40 30 60 80 70 50
```

(Before deletion: In-order 20 30 40 50 60 70 80, Pre-order 50 30 20 40 70 60 80, Post-order 20 40 30 60 80 70 50.)

Q4 b) Iterative traversals using a stack and comparison (13 marks)

```
# uses Node, insert() and the recursive functions from (a)
def inorder_iter(root):
    stack, cur, res = [], root, []
    while cur or stack:
        while cur:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        res.append(cur.data)
        cur = cur.right
    return res

def preorder_iter(root):
    if root is None:
        return []
    stack, res = [root], []
    while stack:
        n = stack.pop()
        res.append(n.data)
        if n.right: stack.append(n.right)
        if n.left:  stack.append(n.left)
    return res

def postorder_iter(root):
    if root is None:
        return []
    s1, s2, res = [root], [], []
    while s1:
        n = s1.pop()
        s2.append(n)
        if n.left:  s1.append(n.left)
        if n.right: s1.append(n.right)
    while s2:
        res.append(s2.pop().data)
    return res

root = None
for v in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, v)

print("In-order  :", inorder_iter(root))
print("Pre-order :", preorder_iter(root))
print("Post-order:", postorder_iter(root))
```

Output:

```
In-order  : [20, 30, 40, 50, 60, 70, 80]
Pre-order : [50, 30, 20, 40, 70, 60, 80]
Post-order: [20, 40, 30, 60, 80, 70, 50]
```

Comparison: The iterative output is **exactly the same** as the recursive output for every traversal. The only difference is the method: recursion uses the system call stack, whereas the iterative method uses our own explicit stack (a list). Iterative avoids the risk of stack overflow for very deep trees.